

The Prover Does Not Choose the Coordinate: One Rule at Four Scales for Anchored Conduct Records

Working paper, draft v0.5, 2026-09-07 (the three departures of Section 7 repaired in gate 0.3.5; the first derived verdicts confirmed by the operator's verifier). Not yet posted.

Toshikatsu Oga, The HORIZON 音s株式会社 (HORIZON SHIELD), Hiratsuka, Japan. ORCID 0009-0000-9180-903X.

Federico Blanco Sánchez-Llanos, Viper Labs (builds invinoveritas, a verification layer for autonomous agents).

[Division of work, stated for the record: T.O. designed and operates the four instances, wrote the harnesses, the anchored specification and its addenda, and drafted this paper. F.B.S.L. is the founding external witness named in those records: his verifier is the counterpart state anchored as entry 25, his review produced the corrections to the time axis, his independent checks of entries 26 to 28 and his question about refusals are in the record. Section 6 is his, written in his own voice from the anchored entries; the witness half of Section 4.3 was drafted by T.O. from the same entries and checked by F.B.S.L. against them. No private message is quoted anywhere in this paper. Both authors approve the final text before posting.]

Abstract

A verifier can be sound on every layer it runs and still return the wrong verdict, because the party being verified was allowed to choose which coordinate got measured; a truthful, reproducible read of the wrong coordinate passes. This paper reports one rule that closes the defect: the verifier derives the coordinate from a source the party being verified does not control and binds the derived value into the verdict as an output a third party recomputes. It shows the rule at four scales in one Bitcoin-anchored record system for agent-facing services: the record (which cost category an estimate is judged against, derived from its own line items), the population (which servers exist to be counted, derived from Certificate Transparency rather than self-declaration), time (when a verdict was created, bracketed by a Bitcoin block-header set the prover cannot choose), and the instant (which day and which tool a conduct measurement samples, derived from a salt committed before the window and a block hash the subject cannot influence). Each instance has a runnable reference implementation, an adversarial harness that holds every sub-layer green while it attacks the coordinate, and an anchored specification whose corrections are appended, never edited. An independent verifier built the counterpart state on his own system, drove three corrections to the operator's time axis, and asked the question that turned a refusal from a sentence in prose into a queryable ledger record; his account is in his own words. The paper also reports what is deployed against what is designed, with the operator as a subject of its own rule: the time check runs from genesis over the peer to peer network on the operator's machine but as a quorum of two among three explorers in the production gate, the census has a live coordinate and no anchored record yet, the record-level guard is a harness not yet wired into the adjudication, and the instant coordinate showed three departures from its anchored design found while this paper was written, one of which the operator's own verifier would have refused; all three were repaired the same day, and the next scheduled sweep produced the first derived verdicts, seven of eight endpoints under one beacon block that the operator's verifier confirmed against a header chain synched from genesis. The limits are named with the claims: determinism is not truth; a quorum buys operator independence, not implementation independence; a refusal proves a contradiction, not which side was honest; the census boundary is structural for private networks and clients that ignore Certificate Transparency.

1. The defect

The record system this paper concerns, NENRIN, is a public, Bitcoin-anchored record of conduct for agent-facing services. Its reproducibility at the ring layer was tested by an independent reimplementation and reported separately (Oga and Blanco Sánchez-Llanos, 2026). That paper established that a stranger to the reference source can rebuild the monthly record byte for byte from the same inputs. This paper is about the failure that byte reproducibility cannot touch: a deterministic, reproducible, correctly anchored read of the wrong thing.

The defect was named at two scales in the same week by the system's founding external witness, and the anchored specification that answers him quotes him at both. At the record level: three verification layers can each be sound and still produce a wrong verdict, because the evidence coordinate is chosen by the party being verified. At the population level: for a census there is no fetch-the-bytes-yourself; a function you never call is not an artifact you can recompute; the best available is naming the undeclared remainder unknown rather than absent.

The two look different. The specification's answer is that they are one defect. A prover supplies a subject and evidence. The verifier checks that the evidence is valid, pinned and reproducible. It never checks that the evidence is about the coordinate the subject actually occupies. At record scale the prover picks which record the evidence reads. At population scale the prover picks

whether it appears in the denominator at all, and a non-appearance leaves no artifact. Every layer is fail-closed. The composition is not.

Two further scales were found after the specification was anchored, one of them on the operator's own instrument. Time is a coordinate: whoever chooses the creation time of a proof, or the block a beacon reads, chooses what the freshness check says. The instant of measurement is a coordinate: whoever can compute the day a conduct gate will sample, and the tool it will call, chooses what the conduct record says. Neither of these is in the specification's original gaming analysis, and the record says so in its own text.

2. Prior art, and where the rule goes past it

None of the components is new, and the anchored documents say so. Certificate Transparency (Laurie, Langley and Kasper, 2013; Laurie, Messeri and Stradling, 2021) and Sigstore Rekord are transparency logs: append-only, publicly auditable, with the involuntary property that a CA logs a certificate whether or not the operator of the host wants it logged. in-toto (Torres-Arias et al., 2019) and SLSA record provenance, who did what to which inputs. OpenTimestamps (Todd, 2016) binds a hash to a Bitcoin block. The record-level coordinate fix that the specification generalises is the witness's own: his review endpoint takes a transaction hash and derives block, contract, status and logs from the receipt, so that the coordinate fields are outputs and asserting a different contract than the transaction touched is a hard failure. The commit-reveal shape used for the instant coordinate is his too, in a different problem: his freshness beacon binds a verdict's timestamp to a future block height the subject cannot have influenced, and it existed before the instant addendum was written. The addendum narrows its claim on that account and names him.

What all of the named prior art shares is that it takes delivery of an artifact the submitting party chose to submit. That is the right design for a certificate, a build, or a signature: the submitter wants the record to exist. It is the wrong design for conduct over time, because conduct is what happens when nobody chose to be watched. The rule reported here is the general form: the coordinate that gets measured is chosen by neither the subject nor the measurer, and the derivation is an output in the verdict that a third party recomputes.

3. One rule

The verifier derives the coordinate from the subject, or from an involuntary channel the prover does not control, and binds the derived coordinate into the verdict as an output. Evidence the prover supplies is accepted only where its coordinate equals the derived one. Mismatch is a hard, named failure. The verdict states in words that the coordinate is derived from the subject and never asserted by the prover.

Two refinements were forced by live systems and are part of the rule as anchored. First, a source the prover does not own is still one point of failure; it can go dark, or lie, and decide alone. So the coordinate is checked against a quorum of independently operated sources, and one reachable source below quorum cannot vouch. Second, a provable lie and an outage are different failures and must not share a bucket. A structurally impossible coordinate fails closed even when other sources are down; a coordinate no reachable source can affirm and none can reject is unverifiable now, named, retryable, and never folded into valid. Fail closed on the adversary, fail open on the outage.

Table 4 states the four instances in one frame.

scale	subject	coordinate	involuntary source	what the prover could choose before	what the verdict binds	refusal
record	an estimate	cost category and scope	the estimate's own line items	the category label	derived category, subject_sha256, the words derived_from_subject	coordinate_chosen_by_prover
population	a scope of servers	membership in the denominator	Certificate Transparency plus an independent probe	whether to appear at all	CT snapshot hash, three sets and a named residual	census discrepancy record
time	a signed verdict	created_at against a beacon	Bitcoin block headers from a quorum of sources, strongest form a locally validated chain	the timestamp, or the beacon read	beacon height and hash, quorum basis, forged vs unverifiable	backdated, forged, bad coordinate
instant	a live endpoint	measurement day and tool	a salt committed before the window plus a block hash at the window boundary	the day (computable) and the tool (list order)	the derivation block, every field an output	derivation refused, legacy fallback disclosed

4. Four instances

4.1 The record: the join guard

The operator's own exposure is a fair-price adjudication. The subject is a construction estimate. The coordinate is the cost category and scope the estimate is judged against. A contractor, the party being verified, can label expensive work as a cheap category. A price-range read of the cheap category is truthful, pinned and reproducible, and the adjudication is wrong.

The guard derives the category from the estimate's own line items, as the most expensive work class actually present, so a cheap label cannot hide an expensive line. The submitted coordinate is accepted only if it equals the derived one. The verdict binds the derived coordinate, states that it is `derived_from_subject` and never `asserted_by_prover`, and binds `subject_sha256`, so that adjudicating a different subject against evidence built for the original coordinate is visible and refused as `coordinate_chosen_by_prover`.

The adversarial harness keeps evidence integrity, canonical form and reproducibility all true in every attack, because that is the point, and shows the join still refuses: a cheaper category, an understated scope, an absent coordinate, and a subject swapped after derivation. Eight vectors, offline, deterministic. The guard is a reference implementation and a harness. It is not wired into the production adjudication path at the time of writing, and Table 2 says so.

4.2 The population: the census

At population scale the source the prover does not control is Certificate Transparency, and the argument that makes it sufficient for the population that matters is short. A server an agent can trust and call is a public HTTPS endpoint. A public HTTPS endpoint that a client enforcing modern PKI norms will connect to presents a CA-issued certificate whose issuance is logged, with a signed certificate timestamp, in Certificate Transparency. The CA logs it; the operator does not choose this. Callability and observability coincide. Every callable public server leaves an involuntary artifact, declared or not.

So the census does not count self-declarations. It recomputes the population from CT and from an independent probe, and reports counts with denominators: declared and callable (on the register and independently confirmed live); declared, no footprint (on the register with no involuntary artifact, a claim with no footprint, flagged and never counted present); undeclared callable (a CT artifact and a live surface, never declared, the set the witness named unknown, here enumerated). The residual, a live server whose certificate is not logged, is named unknown and never absent; an agent under modern PKI norms would not accept such a certificate, so this class cannot be the subject of a trust verdict, and any member a probe does find is listed rather than hidden.

The coordinate is the CT snapshot hash, carried in the record, so a third party re-queries the same log, re-probes, and recomputes the same three sets and the same record hash. A census is not a new object on the ledger. It is a witness whose vantage is an involuntary log, rendered as a `jidec-path-v1` walk, so it rides the existing witness intake with no new server code. The disagreement between two independent censuses of one scope is a census discrepancy record, the same first-class object as the system's founding discrepancy record, one scale up; an operator census that omits a host a second witness finds cannot self-certify complete.

The harness has ten vectors: a certificate without a service (refused: callable-observed requires both a CT artifact and a live probe), CT flooding (inflates undeclared callable and never declared and callable, at real issuance cost and with a permanent artifact), an operator hiding a host (surfaced by any second party's census), and the residual misnamed as absent. The live coordinate is `crt.sh`, a single log front end; a second log source is not wired. No census record has been anchored at the time of writing. Table 2 says so.

The boundary is stated, not worked around. The coincidence holds for callable public servers under clients that enforce SCTs. It does not extend to private-network services, non-HTTPS transports, or clients that ignore SCT enforcement. For those the census marks unknown. The witness's structural wall is real; the specification moves it off the self-declaration line, where it does not belong, and onto the callability line, where it does, and computes everything on the near side.

4.3 Time: the beacon, from a website to a chain

A freshness check binds a verdict's creation time to a public beacon so that the verdict provably existed no earlier than the beacon and no later than a clock. The operator's second version closed backdating with a public beacon and made currency fail-closed. Two seams stayed open in that version and both became concrete on the witness's live system, which read a single explorer, so the operator closed them on its own harness rather than only naming them to someone else.

Seam one, a single source. One source the prover does not own is still one point of failure. Version three checks the beacon against two or more independently operated sources and requires a quorum before it affirms. Seam two, forged versus unverifiable. Version two put a height that provably does not exist on the chain and a height that cannot be checked right now because the source is down in one bucket, and refused both, punishing an honest prover for an outage. Version three separates them: a structurally impossible height is a bad coordinate and fails closed always; a height no reachable source can affirm and none rejects is unverifiable now, not refused, and not asserted current.

Three corrections were made to version three before it was anchored, each found by an outside reader and each reproduced by the operator on the superseded bytes before any change, with the superseded hashes kept on the record. The first: the backdating check had been coupled to the quorum, so a genuinely backdated proof carrying a real historical block passed as indeterminate whenever only one source was reachable, a silent pass. The two checks were decoupled: corroboration still needs a quorum to affirm, but one honest confirmed read is enough to refuse. The second: a structural rejection had been a fixture flag; it is now derived from the sources' own reported tips, with a margin of six blocks, Bitcoin's confirmation depth, so that a lagging source degrades to cannot-confirm and never vetoes a real block. The third, found by the witness's reviewer probing the fix itself: comparing against the highest reachable tip let one source inflate its tip to cover a ghost height and suppress a legitimate reject. The reference tip is now the quorum-th highest reachable tip; a single liar is the highest and is ignored, a single stale source is the lowest and is ignored, and the two-source case, where no such choice exists, is disclosed as degraded rather than hidden. The harness pins each of these: twenty-two checks, including one honest witness refuses, a stale source cannot veto, one inflated tip cannot suppress a reject, a liar buys unknown at most and never authentic, the margin boundary is exact, and near-tip claims converge with the chain. Any mismatch between sources still fails closed, because a contradiction between sources is evidence in itself, and refusing exposes the liar where averaging would hide it.

The sources addendum then said in public what the source list implied and the operator had not yet earned. A quorum buys operator independence, not implementation independence; two explorers running the same codebase are one bug away from two simultaneous faults, and the witness found this on his own set first and wrote it beside his source list. The operator's third source, a locally synced header set, was the strongest form of the check named by the time addendum, and it was a fixture in the harness, not wired to a real header store. Three names did not make three implementations, and the addendum exists partly so that nobody, including the operator, can later read the count as if they did. The same addendum settled the choice of rule for authentic in the open: two of three, the operator's choice, tolerates one source down or lying and opens to two colluders; unanimity, the witness's choice, tolerates two colluders and stops affirming the moment one honest source is unreachable; a dissent rule, the apparent third way, was rejected by both sides, and the number of its window was written down: under it, two colluding sources matching a fabricated height inside the six-block margin would hold authentic for up to those blocks and then flip to forged when the real block lands. What that buys an attacker is a wrong current for under an hour on a proof that then turns forged on the record; it cannot help backdating or postdating. Both rules are on the record with their prices beside them.

The debt was paid in three anchored steps. The first wired the third source to raw eighty-byte Bitcoin block headers, verified by the chain's own consensus rules before any header may say anything about any block: linkage, proof of work against the encoded target, target within the limit, the retarget at every multiple of 2016 recomputed with Bitcoin Core's arithmetic, median time past over eleven timestamps, future drift against the clock, and the genesis hash at height zero. Its adversary does not fake proof of work; it mines real headers at a test difficulty and mines its forgeries too, so that the line between what proof of work refuses and what only a checkpoint refuses is drawn by execution, twenty-five checks. On the real chain, a window of 3,820 headers was pulled from two explorers treated as couriers and not as sources, rebuilt from fields, refused where the rebuilt hash did not equal the courier's id, and validated; the two couriers agreed byte for byte. freshness_v3 was then run offline against three real sources with a fixed clock, and five proofs produced five verdicts whose record hashes reproduce byte for byte from the pinned files, including a proof created a week before its own beacon block, refused as backdated on real data by the decoupling described above. Its stated limit: the courier at sync time was still an HTTP explorer, and a lighter alternative chain was refused only by a checkpoint the operator chose.

The second step made the courier the network. Full nodes were asked for headers over the Bitcoin peer to peer protocol, 182 candidate addresses resolved from eight public DNS seeds, the first three that completed a handshake pulled independently, every header validated by the same seven checks before a byte was written, and the three runs compared byte for byte. There was no website in the path at any time.

The third step removed the checkpoint. On 2026-09-04 at 09:32 UTC the client was run from genesis, the locator computed from the eighty bytes the adapter carries rather than looked up, against three full nodes, in one streaming pass with no checkpoint supplied. 965,457 headers, heights 0 through 965,456, 77,236,560 bytes; 478 retarget boundaries verified from inside the file, none unverified; the whole file validates without a checkpoint in 2.3 seconds; the one disclosure is that the genesis block has no predecessor to be continuous with. The windows the two earlier entries pinned are byte slices of this file at their heights, so the checkpoints those entries chose were confirmed by the bytes and not the other way round. Block 965,447, which confirmed the sources addendum on the Bitcoin chain, is at its offset with the hash the earlier entries reported. The chain that holds the anchor of the record which admitted the third source was a fixture is now held whole, from the first block, by the machine that made the admission. The streaming validator that made the full chain tractable is proven equal to the windowed one by a harness that feeds both the same bytes in random chunk sizes and requires identical reports, eight checks. The 77-megabyte file is not in the repository; its hash and the hashes of four prefixes are pinned so that anyone who syncs from genesis can compare without holding

this copy. What now refuses a lighter alternative chain is the work, and an adversary who can outwork the chain from genesis is not one this specification defends against, and no specification does.

The fourth step was forced by a question from outside. An independent verifier who had recomputed the three preceding entries asked whether the harness case in which two peers serve two valid chains, no checkpoint separates them, and the client refuses and names both, is logged anywhere a downstream consumer of the ledger can query, or whether someone has to read the addendum text to know that the failure mode exists. The honest answer was no. The client had raised an exit with a sentence and written nothing, because the rule "nothing is written on a refusal" had been written for header bytes and had swallowed too much. The client now writes a refusal record on every refusal, before exiting, and still writes no header byte: a JSON record with a schema, a reason code (peers_disagree, below_min_peers, readback_refused, readback_sha_mismatch), the height, both peers by address and both hashes at the first differing offset, every peer that finished with its tip, every peer that was dropped with why, the network stated as mainnet only when both the magic and the parameters are mainnet's, the clock, the peer minimum, and bytes_written false. A seed maker turns the record into a ledger entry in the same shape as every other entry and is itself fail-closed against tampered records. The harness grew to sixteen checks and the two-chains case now requires the record, byte reproducible across machines. That record, from the red team, on a test network with peers on loopback, at height 12, is on the ledger as its own entry, citable by its own hash, so that a consumer can code against the shape of a refusal before any real one has happened. The operator's rule from that entry on is that every refusal against the real network is appended the same way, by the same script, whether or not the operator likes what it says. The record proves that the client saw a contradiction and what it saw. It does not say which peer was honest. That is the point: the client does not choose, and the record does not pretend to.

The production conduct gate cannot open raw TCP connections to Bitcoin nodes from where it runs, so it does not read the locally synced header set. It reads three independently operated block sources and requires two of them to agree on the hash at one reference height, the second-highest reported tip less six, before it treats the result as a beacon, records the height and hash it used so that anyone holding the chain can falsify a wrong beacon permanently, and otherwise falls back to the legacy schedule and says so in the verdict. As deployed until 2026-09-06, each source's height was its own reported tip less six, so two honest sources whose tips differed by one block, which is ordinary, failed to agree; the reference-tip rule that version 3.3 of the harness adopted for exactly this reason was not carried into the production code until 0.3.5 (Section 7). The strongest form of the check runs on the operator's machine and in the harness; the deployed form is the quorum of explorers. Table 2 states both.

4.4 The instant: which day, and which tool

A conduct ring counts instants sampled and instants reached. Whoever chooses the instants chooses what the ring says. In the reference gate as deployed on 2026-09-05, the free tier's measurement day was a bucket computed as the first four hex digits of the SHA-256 of the endpoint URL modulo seven, due when the day count modulo seven equaled the bucket. Every input was public. The target could compute the day it would be measured. The specification's anchored gaming analysis had answered shim farms by saying that a shim that maintains three independent witnesses and an unchanged manifest for eighteen months has paid most of the cost of simply being honest; under a computable schedule the shim need only answer on one day in seven, and the ring cannot tell, because the ring only knows the instants it was handed. The same defect had a second face on the same instrument: the determinism condition measured the first well-formed tool in the server's own tools/list order, and the server chooses the order. The gate had disclosed this as a residual. Disclosing a coordinate defect is not closing it.

How it was found belongs in the record because it is the paper's own mistake. On 2026-09-05 the operator wrote to an outside endpoint that had appeared on the register and, in the name of transparency, published the bucket arithmetic in that letter so the recipient could verify the stated measurement date without trusting the operator. That paragraph was simultaneously an honest disclosure and a working exploit recipe. The letter was not withdrawn, because the arithmetic was already derivable from the public source. The addendum that repairs the defect was published before the code changed, deliberately, so that the defect is on the record while it is still open, and the record says so.

The rule applied to the time axis: derive the coordinate from a source neither party controls, and bind the derivation into the verdict as an output. Two ingredients, and the reason each is necessary is a harness vector rather than an assertion. A salt the measurer commits to before the window and reveals after; without it the target predicts, because every other input is public, and the control vector that holds the salt shows prediction becomes exact. A Bitcoin block height at the window boundary; without it the measurer could mint a salt after measuring and claim whichever instant flattered the result, so the commitment must be anchored below the window's opening height, and a commitment anchored after the window opened is refused both ways. The measured tool is derived by the same salt over the lexicographically sorted tool-name set, so reordering tools/list cannot steer the pick, and renaming a tool to steer it changes the tool-set hash, which a ring already records as a surface change. The derivation block in the verdict carries the schema, the window, the salt commitment, the anchor heights and block hash, the instant slot, the tool-set hash, the tool count and the tool measured, and every field is an output the measurer derived; a third party recomputes all

of it from the revealed salt and trusts nothing the measurer asserted. The beacon that closes backdating is the beacon that closes instant prediction; no new cryptography is introduced, because a scheme nobody can recompute is worse than the defect it replaces. The harness has seventeen vectors: eleven attacks, three controls, one misclassification, two residuals, offline and deterministic; the gate's JavaScript port of the same harness has twenty-six.

Residuals, named and not solved. A salt is single use per window; a revealed salt reused in a later window is fully predictable, and the vector that proves this exists to prove the failure, not to pass. Derivation is fair only inside the surface the subject declared; a tool never listed is never picked, and that set is unknown, not absent, the same boundary the census names at population scale and the witness named first. This does not measure quality; a server that answers honestly at an unpredictable instant with an unpredictably chosen tool may still return nonsense.

The gate implemented the derivation in its 0.3.0 release on 2026-09-05, committed after the addendum's first published version and before the narrowed version that was anchored as entry 33; the gate source carries both hashes and states that the rule is the same in both. Every verdict now carries a `coordinate_derivation` block, and when the beacon is unavailable the verdict records that it fell back to the legacy computable schedule, which the subject can predict, disclosed rather than hidden. Whether it had ever derived was another matter, and the answer found on 2026-09-06 is in Section 7: the first scheduled sweep after deployment had fallen back, the persisted history did not carry the derivation block, and the order of salt and block in the production code was the reverse of the one the addendum requires. The three were repaired the same day, and the sweep of 2026-09-06 at 18:00 UTC produced the first verdicts derived as designed; Table 2 and Section 7 carry the numbers, and Section 9 pre-registers what the rest of September must show.

5. Adversarial harnesses

Every instance is backed by a harness that attacks the coordinate while holding the sub-layers green. Table 1 lists them. Counts are the totals each harness printed when run on 2026-09-06 (all green), and they equal what the anchored documents state where a document states a count; the gate's JavaScript count is the one the operator's claim register measures against the gate's own specification page.

instance	reference implementation	adversary	checks	what stays green in every attack	what is refused
record	<code>join_guard.py</code>	<code>join_redteam.py</code>	8	evidence integrity, canonical form, reproducibility	cheaper category, understated scope, absent coordinate, swapped subject
population	<code>nenrin_census.py</code>	<code>census_redteam.py</code>	10	the CT snapshot hash and the probe	cert without service, CT flooding counted as declared, hidden host, residual misnamed absent
time, probe v1	<code>time_coordinate_probe.py</code>	<code>time_redteam.py</code>	8	real Ed25519 signature, authorship	postdating; backdating and currency named as unsolved by a forward anchor, not claimed caught
time, beacon v2	<code>freshness_v2.py</code>	<code>freshness_v2_redteam.py</code>	9	signature, id integrity	backdating, postdating, stale currency
time, beacon v3.3	<code>freshness_v3.py</code>	<code>freshness_v3_redteam.py</code>	22	signature, id integrity, honest outage never refused	backdated lone source, ghost height, inflated tip suppressing a reject, liar reaching authentic
time, header chain	<code>localheaders.py</code> , <code>localheaders_stream.py</code>	<code>localheaders_redteam.py</code> , <code>stream_redteam.py</code>	25, 8	real proof of work at test difficulty	flipped bit, unmined header, easy bits across a retarget, median time past, future time, duplicate, gap, bad length, manifest mismatch
time, courier	<code>sync_headers_p2p.py</code>	<code>p2p_redteam.py</code>	16	byte agreement across peers	two valid chains (refused and recorded), below minimum peers, tampered refusal record given no seed
instant	<code>instant_coordinate.py</code>	<code>instant_redteam.py</code> (Python), gate port (JavaScript)	17, 26	the revealed salt recomputes every field	salt held, commitment anchored after the window, reordered or renamed tool list, reused salt (residual)

The harnesses are offline and deterministic, and the anchored documents state that each runs identically on the operator's machine and in a clean container. The reader does not have to take the counts from this table: Appendix A gives the commands.

6. The witness's side

[Written by F.B.S.L. Every claim below was independently re-checked against the anchored record before this replaced the placeholder, not written from memory of the exchange.]

My corresponding state is anchored in entry 25 against commit `63fac98cdf2e4bc4985778b65d730d7a419e4d8`, and I re-read that commit's own diff before trusting the record's description of it: every source fetches its own numeric tip on every call, the quorum tip is the second-highest reading, a height-proven fallback lifts a sibling's veto without ever counting toward a match, and authenticity requires unanimity across every configured source, not a bare majority. The shared-codebase disclosure the record cites is mine: two of my three live sources run the same explorer codebase, and I wrote that down beside my own source list on purpose, rather than let a quorum of three read as more independent than it is. The commit is real and current as of that entry; what it does not prove, and what I want stated plainly rather than left implicit, is that the two verifiers are equivalent. Built apart, cross-checked on bytes, currently agreeing on one count, is what the record supports. Nothing more.

Three of the corrections to the time axis trace to that review, and I want the shape of each named precisely rather than folded into one line. The coupled backdating check was a silent pass: a genuinely backdated proof carrying a real historical block would read as merely indeterminate whenever only one source answered, because the two checks shared a gate that should have been separate. The fixture veto replaced a hardcoded rejection flag with one derived from the sources' own reported tips, closing a gap where a lagging source could veto a real block instead of degrading honestly to unresolved. The inflated tip was the sharper of the three, because it came from attacking the fix itself rather than the honest case: a review that only tests the case a fix was built for tells you the fix works when nothing is trying to break it, which is not the same claim as fail-closed. Comparing against the single highest reachable tip let one source inflate its own reading to manufacture a ghost height and suppress a legitimate reject; the correction (the reference tip is the quorum-th highest reachable reading, not the maximum of whatever answers) closes exactly that seam, and I re-verified the corrected logic against the same attack before calling it closed.

I verified the sources and header-chain entries independently rather than take the anchoring on trust: re-hashed the raw bytes myself, deserialized the OpenTimestamps proofs against my own library instead of reusing the operator's, and confirmed the sources addendum resolves to Bitcoin block 965,447, the same block the from-genesis header file holds at its stated offset, with the identical hash, computed the second time from a different starting point. Two independent paths landing on the same block is the actual claim; either path alone would just be an assertion.

The question behind the refusals addendum (entry 31) was a real question, not a rhetorical one: if two peers ever serve two valid but disagreeing chains and no checkpoint exists to separate them, is that failure mode something a downstream consumer of the ledger can query directly, or does it only exist as prose in an addendum someone has to already know to read? The honest answer, at the time I asked it, was that it only existed as prose. Entry 31 is the fix, not a courtesy: a refusal that only a human reading the specification would ever notice is a weaker guarantee than the byte-reproducibility claims sitting right next to it.

My own freshness beacon is the commit-reveal shape the instant addendum narrows its claim against, and I want the provenance stated exactly: I built it to bind a signed verdict's `created_at` to a future Bitcoin block height the subject of the verdict cannot have influenced, for a different, narrower reason, closing a backdating gap on our own `/review` verdicts, before the instant coordinate's gaming analysis existed as a named problem. The addendum is correct that it narrows an existing mechanism to a new use, not that it introduces a new one; crediting it as new would overstate what changed and understate what didn't.

What I have not verified, and won't claim otherwise: the actual contents of the commit named in entry 25 live in my own repository, not in this one, and nothing in this record proves the two verifiers compute the same thing for the right reasons rather than the same thing by coincidence on the inputs checked so far. What the record supports is narrower and, I think, more useful than a claim of equivalence would be: two verifiers, built independently, currently agree.

7. Deployed against designed

The paper's honesty spine is Table 2. Each row says where an instance stands at the time of writing, so that no reader mistakes a harness for a deployment or a deployment for an exercise.

instance	specification	code pinned	deployed in production	exercised on real data	first real exercise expected
record, join guard	entry 22	entry 23	no; the production adjudication does not call it	no	when the guard is wired into the adjudication path; a later entry will cite entry 22
population, census	entry 22	entry 23	live coordinate is crt.sh, a single log front end; no anchored census record	no census record anchored	a first census of the operator's scope, and a second party's census of the same scope
time, beacon v3.3	entry 24, rule settled in entry 26	entry 24	the operator's harness; the production gate uses a quorum of two among three explorers and cannot read the header set	five verdicts on real sources (entry 27), offline with a fixed clock; the production beacon on 2026-09-06 (instant row)	already exercised
time, header chain from genesis	entries 27, 28, 29	entries 27, 28, 29	operator's machine; not in the production gate	yes: 965,457 headers validated, three nodes byte identical	already exercised; re-synced on the freshness cadence
time, refusal record	entry 31	entry 31	client on the operator's machine	one refusal on the ledger, from the red team on a test network (entry 30); none on the real network	the first real refusal, if one occurs
instant coordinate	entry 33	entry 33 (Python), gate 0.3.0 then 0.3.5 (JavaScript)	yes; three departures from entry 33 found on 2026-09-06 and repaired the same day in 0.3.5 (below)	yes: the sweep of 2026-09-06 18:00 UTC wrote seven derived verdicts (mcp, hearing, web, jidec, p001, p002, gate) under beacon block 965808, salt created 2026-09-05T18:00:12Z, block header time 2026-09-06T17:06:00Z; the operator's verifier reports MATCH, 0 falsified, against a header chain synced from genesis	the eighth row (femtech) at the next sweep; every later window on schedule (Section 9)

Three rows of that table were changed by the act of writing it. On 2026-09-06 the first author re-exported the gate's public history for all eight endpoints and ran the operator's own beacon verifier over them, expecting to report the first derived verdicts from the 2026-09-05 18:00 UTC sweep, the first scheduled sweep after gate 0.3.0 was deployed. There were none, and each reason is a departure from entry 33. First, the persisted history entry does not carry the derivation block at all: the verdict object the gate returns to a caller carries it, but the function that condenses a verdict into a history entry copies status, reachability, the record hash, consent, conditions and surface, and nothing about the coordinate, so the export that feeds the rings, the public archive and the verifier is structurally silent on whether any instant was ever derived, and the claim register's check passed by finding nothing to check. Second, that first sweep did not derive at all; it fell back to the legacy schedule, as the gate's own sweep summary records, because agreement is demanded on a height each source computes from its own tip, and two honest explorers one block apart never agree, the seam the harness closed in versions 3.2 and 3.3 and the production code did not inherit. Third, the production code creates the window's salt lazily at the first sweep of the window and, in the same call, reads a beacon six blocks below the current tip, a block mined about an hour before the salt existed; the addendum requires the commitment to be anchored below the window's opening height and the beacon to be a block mined after the salt, the operator's verifier implements that order and marks the reverse as falsified with the words that the gate could have chosen the salt knowing the hash, so had the sweep derived, the operator's own verifier would have refused the result. None of this changes the design, all of it changes the row, and the repairs are three: persist the derivation block in the history entry; agree on a hash at one reference height rather than on per-source heights; and create the salt at the window's opening and bind only to a block mined after it, publishing the commitment when it is made. The operator is a subject of the rule, and this is what that costs: the paper reports the deployment it found, not the one it meant to describe.

The repairs shipped the same day as gate 0.3.5. The history entry now carries the derivation block. The beacon is read from three independently operated sources at one reference height, the second-highest reported tip less six, and agreement is demanded on the hash at that height, not on per-source heights. The salt for a window is created before the window opens, a beacon is accepted only when the block's header time is later than the salt, and the window's commitment is published on the gate when it is made. The next scheduled sweep, 2026-09-06 at 18:00 UTC, derived. Seven of the eight rows (mcp, hearing, web, jidec, p001, p002, gate) were measured under window w2957 with beacon block 965808, hash 00000000000000000217ec231db790d8ffea4328d608f7e4c7a56736d08eb1, header time 2026-09-06T17:06:00Z; the salt had been created at 2026-09-05T18:00:12Z, twenty-three hours before the block, and its commitment ff57ab419114d900da6d74f6a7cdc475e04c78435521303497dd862a1bf4b733 had been published at creation. The operator's own verifier, run on 2026-09-07 against a header chain synced from genesis over the peer to peer protocol, three peers byte identical over 965,850 headers, reports that beacon as MATCH, the chain's block at that height with the salt before it, 0 falsified. The eighth row, femtech, was not measured that day. The gate runs within a budget of 50 outbound fetches per invocation; the first sweep of a window spends part of that on the three beacon sources and on filing the commitments before any row is measured, and on 2026-09-06 that left room for seven rows, 47 of 50 fetches used. The sweep record lists the row as skipped and says why, and it goes first at the next sweep. A skipped row is an instant not sampled, disclosed; a measurement cut off midway would have been a false instant, and the fetch count that prevents it was added the same evening, before the sweep. At the same sweep the commitment of the following window, w2958, was filed to the ledger's witness intake, three days before that window opens on 2026-09-10 at 00:00 UTC; its Bitcoin attestation is pending at the time of writing, and Section 9 pre-registers

the test. From that window on every commitment is anchored before its window opens; w2957 is the last whose commitment has only the gate's own page as evidence of when it was made.

8. Limits

Determinism is not truth. Every harness here proves that a coordinate cannot be chosen by the prover without refusal. None proves that the measurement taken at the derived coordinate is accurate. That belongs to the witness layers, and the August rings carry one witness.

A quorum buys operator independence and not implementation independence. The three public explorers in the operator's production quorum are independently operated, and two of them run the same explorer codebase, the axis the sources addendum names, so the quorum's implementation independence rests on one source. The locally validated header chain is the source that is a different implementation by construction, and it runs where the production gate cannot.

A refusal is a record, not a detection. It proves that the client saw a contradiction and what it saw. It does not say which peer was honest, and a client that pretended to know would be choosing the coordinate itself.

The census boundary is structural for private-network services, non-HTTPS transports, and clients that ignore SCT enforcement. Those are marked unknown, never absent. If such a class ever becomes trust-relevant, a new involuntary enumerator must be named for it, or the boundary is structural there.

The instant coordinate's salt is single use per window; a leaked salt burns one window only if that rule holds. Derivation is fair only inside the surface the subject declared.

The gate's measurements are bounded by a budget of outbound fetches per invocation, and the first sweep of a window spends part of it on the coordinate itself before any row is measured. A row that does not fit is skipped and the sweep record says so; the alternative, a measurement cut off midway, would be a false instant rather than a missing one. Which rows fit on a given day is a fact about the deployment's budget, reported as such, not a property of the design.

The operator is a subject, not an exception. Its own endpoints are counted under the same census rules; a host it runs but did not declare appears in undeclared callable against its own census. The join guard applies to its own adjudications. The defect in the instant coordinate was the operator's own, disclosed by the operator in a letter that doubled as an exploit, and repaired on the record.

Two things this paper does not claim. It does not claim that any of the four instances is the only way to close the defect at its scale. It does not claim that the rule closes coordinates nobody has named yet; the time and instant scales were found after the specification was anchored, and the record expects a fifth.

9. Pre-registration

Two things the September 2026 records will show for the first time, and either outcome is reported in a revision of this paper. First, derived verdicts: the first seven exist, dated 2026-09-06 18:00 UTC, and the operator's verifier confirms them (Section 7). For the rest of September the exports either carry coordinate_derivation blocks whose beacon the verifier confirms, or they carry the legacy fallback disclosed; the count of each, per window, is reported as measured. Two further tests are fixed now: the eighth row (femtech) is measured under a derived coordinate at the next sweep, and the commitment for window w2958, filed to the ledger on 2026-09-06, is carried by a ledger entry whose Bitcoin attestation predates that window's opening on 2026-09-10 at 00:00 UTC, so that the salt's creation is bounded from above by the chain and not by the gate's own page. Either outcome of each is reported. Second, a census: the operator will run its own census of the scope it operates and invite a second party to run one of the same scope; either the two agree, or the first census discrepancy record is anchored.

Appendix A. Reproduce it yourself

Clone github.com/ogasurfproject-jpg/horizon-shield and enter `workers/hs-ledger/nenrin/coordinate-v1`. All harnesses run offline and deterministically:

```
python3 join_guard.py ; python3 join_redteam.py
python3 nenrin_census.py ; python3 census_redteam.py
python3 time_coordinate_probe.py ; python3 time_redteam.py
python3 freshness_v2.py ; python3 freshness_v2_redteam.py
python3 freshness_v3.py ; python3 freshness_v3_redteam.py
python3 localheaders_redteam.py ; python3 stream_redteam.py
python3 p2p_redteam.py
python3 instant_redteam.py
```

The live census coordinate is `python3 nenrin_census.py --live` against `crt.sh`. The header chain is not in the repository; sync it from genesis with `sync_headers_p2p.py` and compare the prefix hashes pinned in the `localheaders v3` addendum. Each anchored document is checked by taking its SHA-256, comparing it with the claim of its ledger entry at `ledger.horizonshield.dev/ledger/{n}?format=raw`, and verifying the entry's OpenTimestamps proof at `ledger.horizonshield.dev/ledger/{n}/ots`.

Appendix B. The anchored entries

Table 3 lists the ledger entries this paper rests on, with the SHA-256 of each anchored document as recomputed from the repository on 2026-09-06; all eleven values equal the claims of the corresponding ledger entries. Bitcoin block heights were read from the ledger on 2026-09-06, when every entry was confirmed.

entry	schema	anchors	claim SHA-256	Bitcoin block
22	nenrin-coordinate-v1	the specification, two scales	5be2b22e339d8b5c45a272325c49da189f10715b01683025ae903e83bf251df5	965312
23	nenrin-coordinate-v1-manifest	pinned bytes of the eight harness files	24a7ed47167dc068f5bd06d94cc47c65f7e5af938efd740d288d45301181586b	965333
24	addendum time-v3	quorum; forged versus unverifiable; corrections v3.1, v3.2, v3.3	447bcf4f38cd8099683ccd396467609438aa47399e9bb9b75d7c425900147611	965333
25	nenrin-witness-state-0001	the founding witness's corresponding state	950edfee4e57835f7bdf7f22e07c54392fad692ad88f5c89aacfee5cdf8a64b4	965345
26	addendum sources-v1	operator independence, outside corroboration, the rule for authentic	04908cd53c006ea0ebd24535ab8d3c884317cd2aeeff7a24f136c8a459cf50dc	965447
27	addendum localheaders-v1	a chain and not a website	f5512ea3bb476e3356f96979c9a922e102f35d893659d76ad151fed5450f6162	965458
28	addendum localheaders-v2	the courier is the network	5ed2027f4ce46a11a35dc065c74b02b81a4289c759225776f9383f4909cdf5f4	965458
29	addendum localheaders-v3	from genesis, no checkpoint	937ce7049c1962f9e862af5f124ae31a40ef39c230a5c0937a1a23257a861693	965458
30	nenrin-localheaders-refusal-1	a red team refusal record, test network, two valid chains at height 12	07830db2cf32aade74b77c3cc563318515d276a505288ae574a3666f7defcf26	965464
31	addendum refusals-v1	a refusal is a record	71683e3a7b44bda0f55f6aefbf0e34280d05dc15f45eb36b8b128fe61bf469b5	965464
33	nenrin-instant-v1	the instant coordinate; the narrowing after the witness's freshness beacon	e228dfd8e5353525fb11cc245be48cff7ef6d01433b27f472f5fc8bb6de39c7d	965566

The refusal record of entry 30 is reproduced verbatim, because it is the one machine-readable artifact in this paper whose whole point is that a consumer can code against it:

```
{
  "bytes_written": false,
  "chain": "test",
  "checkpoints": {},
  "courier": "bitcoin p2p getheaders",
  "disagreement": {
    "common_headers_before": 12,
    "hash_a": "00018351cef26ed0f846d1d0bfce7bcea0163af2ce1cb142679645ee5ba2a371",
    "hash_b": "0001ea441405948b1361460320f0db2f617ac2a4807ea112336b324fc9c52",
    "height": 12,
    "peer_a": "127.0.0.1:28901",
    "peer_b": "127.0.0.1:28902"
  },
  "height": 12,
  "min_peers": 2,
  "mode": "window from manifest",
  "network": "test network (magic 0b11fa09, params test): not the real chain",
  "note": "a refusal is evidence. no header bytes were written; this record was. a contradiction between sources i
s exposed here by name and height so that anyone can query it, not only read about it.",
  "peers_failed": {},
  "peers_finished": {
    "127.0.0.1:28901": {
      "headers": 32,
      "tip_height": 31
    }
  },
  "peers_wanted": 2,
  "reason": "peers disagree over the common window: 127.0.0.1:28901 vs 127.0.0.1:28902 at height 12.",
  "reason_code": "peers_disagree",
  "refused_at": 1700100000,
  "schema": "nenrin-localheaders-refusal-1",
  "start_height": 0
}
```

Acknowledgements and disclosure

The first author operates every endpoint, harness and ledger cited in this paper; that is the self-application the specification requires, and it is also a conflict of interest, stated here. The second author operates a separate verification service, *invinoveritas*, whose state is anchored as entry 25, and is the founding external witness named in the anchored records; where this paper reports his findings it cites those records. The two authors are collaborating on a separate open project, an adapter for the semantic-abi type system, and no payment has passed between them in either direction. No party paid for any measurement, harness run, or record cited.

References

- Laurie, B., Langley, A., Kasper, E. (2013). Certificate Transparency. RFC 6962, IETF. <https://www.rfc-editor.org/rfc/rfc6962>
- Laurie, B., Messeri, E., Stradling, R. (2021). Certificate Transparency Version 2.0. RFC 9162, IETF. <https://www.rfc-editor.org/rfc/rfc9162>
- Sigstore project. Rekor: a transparency log for software supply chain signatures. <https://github.com/sigstore/rekor>
- Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., Cappos, J. (2019). in-toto: Providing farm-to-table guarantees for bits and bytes. Proceedings of the 28th USENIX Security Symposium, 1393 to 1410.
- SLSA: Supply-chain Levels for Software Artifacts, version 1.0 (2023). <https://slsa.dev>
- Todd, P. (2016). OpenTimestamps: Scalable, Trust-Minimized, Distributed Timestamping with Bitcoin. <https://opentimestamps.org>
- Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. <https://bitcoin.org/bitcoin.pdf>
- Oga, T., Blanco Sánchez-Llanos, F. (2026). Same File In, Same Bytes Out: Reproducible Conduct Records for Agent-Facing Services, Tested by an Independent Reimplementation. SSRN working paper 7419998. <https://ssrn.com/abstract=7419998>
- Oga, T. (2026). NENRIN Coordinate Integrity, at two scales (NENRIN_COORDINATE_SPEC_v1.md), JIDEC entry 22, SHA-256 5be2b22e339d8b5c45a272325c49da189f10715b01683025ae903e83bf251df5, and its addenda, JIDEC entries 23 to 31 and 33. github.com/ogasurfproject-jpg/horizon-shield, [workers/hs-ledger/nenrin/coordinate-v1/](https://github.com/ogasurfproject-jpg/horizon-shield).
- Oga, T. (2026). NENRIN v1: Machine-Readable Tree Rings for Agent-Facing Services (NENRIN_SPEC_v1.md), SHA-256 9ccb a2e325fd2a555fcdb2dec519b8c6bf7a669064674846aea98ecff824e3d. github.com/ogasurfproject-jpg/horizon-shield, [workers/hs-ledger/nenrin/](https://github.com/ogasurfproject-jpg/horizon-shield).
- JIDEC ledger, HORIZON SHIELD. <https://ledger.horizonshield.dev/ledger/{n}>, raw bytes at `?format=raw`, OpenTimestamps proof at `/ots`.